Monster High
Institute Manager

Final Programming Project

1ºDAM

IES Nervión

Fátima Román

2025–2026

May 2026

School Management System

Contents

1. Introduction and System Description

1.1. About the project

Monster High Institute Manager is a Java desktop application for managing a fictional school inspired by the *Monster High* franchise. I chose this theme on purpose: I wanted to work on something I actually liked, and Monster High gave me the motivation to keep going through the harder weeks of the project.

From a technical point of view, the application lets you manage everything in a school: students, teachers, subjects, groups, enrolments, timetables and the monster types that students belong to. It has a graphical interface built with JavaFX, and data is saved both in CSV files and in a SQLite database.

1.2. Who is it for?

The application is aimed at the administrative staff of a fictional school. With it, they can manage:

- Enrolled students and their academic data.
- Teachers and the subjects they teach.
- Class groups (*MonsterHighGroups*) and their timetables.
- Student enrolments (*Enrollments*) in each subject.

1.3. Functional scope

The application provides a full CRUD (create, read, update, delete) for every main entity. It also includes more advanced features: filtering by teacher specialty, searching students by surname, grade statistics, and grouping by monster type.

2. Analysis and Design

2.1. Main design decisions

Before writing any code, I drew the UML diagram to have a clear idea of the architecture. The most important decisions I made were:

1. **Three-layer architecture**: the application is split into presentation, service, and data access layers. This means the business logic does not depend on whether data comes from a CSV or from a database.
2. **DAO pattern**: each entity has its own Data Access Object that extends a generic abstract class called `GenericRepositoryBD<T>`. This avoids repeating code and makes adding a new entity straightforward.
3. **Singleton for the connection**: the `DatabaseConnection` class is a Singleton, which guarantees that the whole application only ever uses one single connection to SQLite.

4. **Hybrid persistence strategy**: data is always saved in CSV files, which act as the portable source of truth. When the application starts, the CSVs are loaded and pushed into SQLite, which is then used as the query engine during the session. This approach came from the need to meet both project requirements (files and JDBC) at the same time. It is not the most efficient architecture, but it works correctly and data is never lost between sessions.
5. **JavaFX over console**: a working console interface was built first, and then JavaFX was added as the graphical presentation layer.

2.2. Package architecture

Package	Contents
`model`	Domain classes: `Person`, `Student`, `Teacher`, `Subject`, `Enrollment`, `MonsterType`, `MonsterHighGroup`, `Schedule`
`repository`	Generic abstract class `GenericRepositoryBD<T>` and the entity-specific DAOs
`service`	Business logic layer: `StudentService`, `TeacherService`, `SubjectService`, `EnrollmentService`, `MonsterTypeService`
`ui`	JavaFX controllers and the console interface
`exceptions`	Custom domain exceptions
`util`	`DatabaseConnection` (Singleton) and `CsvUtil`

2.3. Why these collections?

Each collection was chosen for a specific reason, not randomly:

Collection	Where	Why
`List<T> (ArrayList)`	DAOs, services, query results	Insertion order matters ; frequent index access
`Map<K,V> (HashMap)`	`MonsterTypeService.groupByType()`	Grouping students by monster type; O(1) key lookup
`Optional<T>`	ID searches in services	Avoids `NullPointerException` when a search may return nothing
`Map<String,Double>`	Grade statistics	Fast name-to-grade association for Stream calculations

3. Object-Oriented Programming

3.1. Class hierarchy

The core of the domain model starts with an abstract class called `Person`, which `Student` and `Teacher` both extend. This hierarchy makes real semantic sense: students

and teachers are both people, and they share attributes like name, surname, date of birth and email.

```
public abstract class Person implements Identifiable,
   Evaluable {
    private int id;
    private String firstName;
    private String surname;
    private String birthday;
    private String email;

    // Constructor with parameters
    public Person(int id, String firstName, String surname,
                  String birthday, String email) { ... }

    @Override
    public abstract String getRoleDescription();

    @Override
    public String toString() { ... }

    @Override
    public boolean equals(Object o) { ... }

    @Override
    public int hashCode() { ... }
}
```

Listing 1: Abstract class Person (simplified)

3.2. Encapsulation

All attributes in every class are `private`. Access is only possible through getters and setters, which include basic validations (for example, name cannot be null or empty, and grades must be between 0 and 10).

3.3. Polymorphism

Polymorphism is mainly shown through `Person` references that point to `Student` or `Teacher` objects at runtime:

```
List<Person> persons = new ArrayList<>();
persons.add(new Student(...));
persons.add(new Teacher(...));

for (Person p : persons) {
    // getRoleDescription() is resolved at runtime
    System.out.println(p.getRoleDescription());
}
```

Listing 2: Polymorphism with Person references

4. Inheritance and Abstract Classes

4.1. Full hierarchy

The inheritance hierarchy has two levels with real meaning:

`Person` (abstract) ⟶ `Student`, `Teacher`

There is also a parallel hierarchy in the repository layer:

`GenericRepositoryBD<T>` (abstract) ⟶ `StudentDAO`, `TeacherDAO`, `SubjectDAO`, `EnrollmentDAO`, `MonsterTypeDAO`, `MonsterHighGroupDAO`

4.2. GenericRepositoryBD abstract class

This generic abstract class defines the contract that all DAOs must follow, so common code is not repeated in each one:

```
1 public abstract class GenericRepositoryBD<T> {
2
3     protected abstract void save(T entity) throws
          DatabaseException;
4     protected abstract void update(T entity) throws
          DatabaseException;
5     protected abstract void deleteById(int id) throws
          DatabaseException;
6     protected abstract T findById(int id) throws
          DatabaseException;
7     protected abstract List<T> findAll() throws
          DatabaseException;
8 }
```

Listing 3: GenericRepositoryBD (simplified)

Each concrete DAO implements these methods with the SQL statements for its own entity.

5. Interfaces

The project defines four custom interfaces that model behavioural contracts:

Interface	Contract it defines
`Identifiable`	`getId(): int` and `setId(int): void` — any entity with a unique identifier
`Evaluable`	`calculateFinalGrade(): double` and `hasPassedAll(): boolean` — entities on which a grade can be calculated
`Exportable`	`toCsv(): String` — entities that can be serialised to CSV format
`Searchable`	`matchesKeyword(String keyword): boolean` — entities that support keyword search

Interfaces are used as types in lists and method parameters. For example, `CsvUtil` receives `List<Exportable>` so it can write any entity without knowing its concrete type:

```
public static void exportToCSV(List<? extends Exportable>
   entities,
                               String path) throws
                                  IOException {
    try (BufferedWriter bw = new BufferedWriter(new
       FileWriter(path))) {
        for (Exportable e : entities) {
            bw.write(e.toCsv());
            bw.newLine();
        }
    }
}
```

Listing 4: Exportable used as a type in CsvUtil

6. Collections and Generics

6.1. Custom generic class

In addition to `GenericRepositoryBD<T>`, the project includes a generic utility method in `CsvUtil` that works with any exportable type:

```
// In StudentService, using the typed DAO:
public List<Student> findAll() {
    return studentDAO.findAll(); // typed List<Student>
}

// Generic method in CsvUtil:
public static <T extends Exportable> void export(List<T> list,
                                                 String path)
                                                    { ... }
```

Listing 5: Real use of a custom generic

6.2. Streams and lambda expressions

Streams are used in the service layer for all query and statistics operations. Here are five real uses, each solving an actual need in the system:

```java
// StudentService: filter students who have passed all subjects
public List<Student> findPassedStudents() {
    return studentDAO.findAll().stream()
        .filter(Student::hasPassedAll)
        .collect(Collectors.toList());
}
```

Listing 6: filter() — students who have passed

```java
// Get the full names of all teachers
public List<String> getAllTeacherNames() {
    return teacherDAO.findAll().stream()
        .map(t -> t.getFirstName() + " " + t.getSurname())
        .collect(Collectors.toList());
}
```

Listing 7: map() + collect() — list of names

```java
// Sort students alphabetically by surname
public List<Student> findAllSortedBySurname() {
    return studentDAO.findAll().stream()
        .sorted(Comparator.comparing(Student::getSurname))
        .collect(Collectors.toList());
}
```

Listing 8: sorted() with a lambda Comparator

```java
// MonsterTypeService: group students by their monster type
public Map<String, List<Student>> groupStudentsByMonsterType()
   {
    return studentDAO.findAll().stream()
        .collect(Collectors.groupingBy(
            s -> s.getMonsterType().getName()
        ));
}
```

Listing 9: groupingBy() — students by monster type

```java
// EnrollmentService: calculate the average grade for a group
public double averageGradeByGroup(int groupId) {
    return enrollmentDAO.findByGroup(groupId).stream()
        .mapToDouble(Enrollment::calculateFinalGrade)
        .average()
        .orElse(0.0);
}
```

Listing 10: average() — group average grade

7. Exception Handling

7.1. Custom exceptions

Six custom exceptions were created for the domain logic, all placed in the `exceptions` package:

Exception	Type	When it is thrown
`StudentNotFoundException`	`RuntimeException`	A student cannot be found by ID
`TeacherNotFoundException`	`RuntimeException`	A teacher cannot be found by ID
`SubjectNotFoundException`	`RuntimeException`	A subject cannot be found
`DuplicateEnrollmentException`	`Exception`	A student is already enrolled in that subject
`InvalidMonsterTypeException`	`Exception`	An unrecognised monster type is used
`GradeOutOfRangeException`	`Exception`	A grade is outside the 0–10 range
`DatabaseException`	extends `RuntimeException`	A database access error occurs

7.2. try-catch in practice

```
public void enrollStudent(int studentId, int subjectId)
        throws DuplicateEnrollmentException {
    try {
        if (enrollmentDAO.existsByStudentAndSubject(studentId,
           subjectId)) {
            throw new DuplicateEnrollmentException(
                "Student " + studentId +
                " is already enrolled in subject " +
                   subjectId);
        }
        enrollmentDAO.save(new Enrollment(studentId,
           subjectId));
    } catch (DatabaseException e) {
        System.err.println("[ERROR] Database failure during
           enrolment: "
                           + e.getMessage());
    }
}
```

Listing 11: Exception handling in EnrollmentService

8. Persistence

8.1. CSV + SQLite hybrid strategy

The application uses a double persistence strategy that works like this:

1. **On startup**: `CsvUtil` reads the CSV files from the `recursos/` folder and loads all data into memory. Then `DatabaseConnection` initialises SQLite and pushes that data into the tables (which are recreated every time the app starts).
2. **During the session**: all read and write operations work against SQLite, taking advantage of its query speed.
3. **On close** ("Exit" option in the menu): all data in memory is exported back to the CSV files via `CsvUtil`, making sure the next session starts with the latest data.

This architecture came from needing to meet both requirements (CSV files and JDBC) at the same time. I know the cleaner solution would be to use the database as the only persistent source and CSVs just for import/export. But this approach works correctly and no data is lost between sessions, so it does what it needs to do.

8.2. CSV file paths

All paths are centralised in `CsvUtil` as static constants:

```
public class CsvUtil {
    public static final String STUDENTS_PATH  =
       "recursos/students.csv";
    public static final String TEACHERS_PATH  =
       "recursos/teachers.csv";
    public static final String SUBJECTS_PATH  =
       "recursos/subjects.csv";
    public static final String MONSTERS_PATH  =
       "recursos/monster_types.csv";
    public static final String EXPORT_PATH    =
       "recursos/export/";
    // ...
}
```

Listing 12: Path constants in CsvUtil

8.3. DatabaseConnection — Singleton pattern

```
public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;
    private boolean schemaInitialized = false;

    private DatabaseConnection() {
        try {
            String url = "jdbc:sqlite:" + DB_URL;
            connection = DriverManager.getConnection(url);
        } catch (SQLException e) {
            throw new DatabaseException("Could not connect to
               SQLite", e);
```

```
        }
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public Connection getConnection() {
        return connection;
    }
}
```

Listing 13: Singleton DatabaseConnection

8.4. DAO example with PreparedStatement

All DAOs use `PreparedStatement` to prevent SQL injection and handle resource closing with try-with-resources:

```
@Override
public void save(Student student) throws DatabaseException {
    String sql = "INSERT INTO students(id, firstName, surname, " +
                 "birthday, email, studentFloor, monsterType) " +
                 "VALUES (?, ?, ?, ?, ?, ?, ?)";
    try (PreparedStatement ps =
             DatabaseConnection.getInstance()
                               .getConnection()
                               .prepareStatement(sql)) {
        ps.setInt(1, student.getId());
        ps.setString(2, student.getFirstName());
        ps.setString(3, student.getSurname());
        ps.setString(4, student.getBirthday());
        ps.setString(5, student.getEmail());
        ps.setInt(6, student.getStudentFloor());
        ps.setString(7, student.getMonsterType().getName());
        ps.executeUpdate();
    } catch (SQLException e) {
        throw new DatabaseException("Error saving student: "
                                    + e.getMessage(), e);
    }
}
```

Listing 14: StudentDAO.save() with PreparedStatement

9. Design Patterns

9.1. Singleton — DatabaseConnection

Problem it solves: SQLite is a single file; opening multiple connections at the same time can cause locking issues and inconsistent behaviour. The application needs to guarantee that only one connection ever exists.

How it works: `DatabaseConnection` has a private constructor and exposes a static `getInstance()` method that creates the instance the first time and returns it on every call after that.

Real benefit: all DAOs get the connection by calling `DatabaseConnection.getInstance().getConnection()`, and none of them ever create their own connection.

9.2. DAO — Data Access Object

Problem it solves: the business logic (services) should not know anything about how data is stored, whether in SQLite, CSV or in memory. Changing the persistence system should not require touching the service code.

How it works: each entity has a DAO that extends `GenericRepositoryBD<T>`. That abstract class defines the common methods (`save`, `update`, `deleteById`, `findById`, `findAll`), and each concrete DAO only adds the methods specific to its entity.

Real benefit: `StudentService` calls `studentDAO.findAll()` without knowing if the data comes from SQLite or a file. The separation of responsibilities is clear.

10. JavaFX Graphical Interface

10.1. Environment setup

The project was developed with **Eclipse IDE**. JavaFX was added manually by placing the libraries in the project's `lib/` folder and configuring the JavaFX modules in the VM arguments:

```
--module-path lib/javafx-sdk/lib
--add-modules javafx.controls,javafx.fxml
```

Listing 15: VM arguments for JavaFX

Maven and Gradle were not used. Dependencies were managed manually.

10.2. Implemented views

The graphical interface has more than three functional views:

View	Functionality
Main screen	Navigation menu to all sections
Student management	`TableView` with full CRUD, surname search and filter by monster type
Teacher management	`TableView` with CRUD and filter by specialty
Subject management	Subject list with teacher assignment
Enrolments	Enrolment form with duplicate validation
Timetables	Timetable view by group

10.3. JavaFX controls used

- `TableView` with `TableColumn` and `CellValueFactory` for data tables.
- `TextField`, `ComboBox`, `DatePicker` for input forms.
- `Button` with event handlers (`setOnAction`).
- `Alert` for confirmation and error messages.
- `Label` for contextual information.

11. Console Interface

Before building the JavaFX interface, the project had a fully working console menu with this structure:

```
===== MONSTER HIGH INSTITUTE MANAGER =====
1. Student management
2. Teacher management
3. Subject management
4. Group management
5. Enrolments
6. Timetables
7. Statistics
0. Exit (saves data)
```

Each submenu provides full CRUD. Input validation prevents the program from crashing on incorrect entries: numeric inputs are validated with `try-catch` on `NumberFormatException`, and empty strings are checked before being processed.

12. Javadoc

All public classes, interfaces, attributes and methods are documented with Javadoc comments, following the standard with tags `@author`, `@version`, `@param`, `@return` and `@throws`:

```
1 /**
2  * Business logic service for student management.
```

```
 * Coordinates operations between the presentation layer and
    the DAO.
 *
 * @author F tima Rom n
 * @version 1.0
 */
public class StudentService {

    /**
     * Finds all students whose surname matches the given
        string
     * (partial, case-insensitive search).
     *
     * @param surname Surname or fragment to search for.
     * @return List of matching students.
     * @throws StudentNotFoundException if no student is found.
     */
    public List<Student> findBySurname(String surname)
            throws StudentNotFoundException { ... }
}
```

Listing 16: Javadoc example in StudentService

The HTML documentation is generated with:

```
javadoc -d doc/ -sourcepath src/ -subpackages
model:service:repository:exceptions:util
```

13. Difficulties and Solutions

13.1. JDBC: two days stuck

Without a doubt, the hardest part of the whole project was getting JDBC to work. At the beginning I did not understand what a connection string was, why SQLite needed a specific driver or how to link it to the project without Maven.

I spent two days where the program simply would not start, or it threw a `ClassNotFoundException` that I did not know how to fix. The solution came from combining the official SQLite-JDBC documentation, YouTube videos and a lot of trial and error. In the end I got it working by adding the `sqlite-jdbc.jar` driver manually to the Eclipse build path and setting the connection URL correctly.

I will be honest: at this point I cannot explain every single line of the connection code in detail. But I know what each block does, I know it works, and I can change it if something needs to change. That was also something I learnt: sometimes you get it working first and fully understand it later.

13.2. JavaFX and module configuration

Setting up JavaFX in Eclipse without Maven was another difficult point. Module errors (`module not found`, `package not visible`) are not intuitive when you are still

learning. The solution was to follow the OpenJFX documentation step by step and understand that JavaFX from Java 11 onwards is an external library that has to be added manually.

13.3. The CSV + SQLite strategy

When I was trying to meet both persistence requirements (files and database) without knowing how to properly combine them, I came up with the approach of reloading the database from CSV on every startup. It is not the standard way to do it, but it solves the problem and data persists correctly between sessions.

14. User Manual

14.1. Requirements

- Java 17 or higher installed.
- JavaFX SDK downloaded and placed in the `lib/javafx-sdk/` folder.
- The `sqlite-jdbc.jar` file in the `lib/` folder.
- Eclipse IDE (or any Java IDE).

14.2. How to run the project

1. Import the project into Eclipse as an *Existing Java Project*.
2. Check that the libraries in `lib/` are on the Build Path.
3. Add the VM arguments in *Run Configurations*:

   ```
   --module-path lib/javafx-sdk/lib
   --add-modules javafx.controls,javafx.fxml
   ```

4. Run `Main.java`.
5. The application will automatically load data from the CSV files in `recursos/`.

14.3. Basic usage

14.3.1. Adding a student

From the main menu, go to *Student management* → *New student*. Fill in the form with name, surname, email, floor and monster type. Click *Save*. The student will appear in the table straight away.

14.3.2. Enrolling a student

Go to *Enrolments* → *New enrolment*. Select the student and subject from the dropdowns. The system will automatically check whether that enrolment already exists and show a warning if it does.

14.3.3. Checking statistics

From the main menu, go to *Statistics*. It shows the average grade by group, the student ranking and the distribution by monster type.

14.3.4. Closing the application

Always close using the *Exit* button in the main menu (or option 0 in the console). This makes sure data is saved to the CSV files before the application ends.

15. Conclusions

This project has been the most complete experience of the whole year. For the first time I had to design a software architecture from scratch: decide which classes I needed, how they relate to each other, what each layer does and why.

What surprised me the most was that the concepts that seemed abstract in class, like inheritance, polymorphism and design patterns, start making sense when you apply them to a real problem. When I saw that the DAO pattern let me change the data source without touching the service code, I finally understood why that pattern exists.

The two things I am most proud of are: getting JDBC to work after two days of frustration, and seeing the final result of the graphical interface. There is something very satisfying about opening an application you built yourself that has its own visual personality.

If I could improve something, it would be the persistence strategy: I would make the database the only source of truth and use CSV just for export. I would also add unit tests with JUnit 5, which I did not have time to implement.

To summarise, this project taught me that programming well is more than making the code work. It is about designing it so that it can be understood, maintained and extended.

16. References

1. **Java 17 Official Documentation** — https://docs.oracle.com/en/java/javase/17/
2. **SQLite JDBC Driver** — https://github.com/xerial/sqlite-jdbc
3. **OpenJFX — Getting Started with JavaFX** — https://openjfx.io/openjfx-docs/
4. **Baeldung — Java Streams** — https://www.baeldung.com/java-8-streams
5. **Refactoring Guru — Design Patterns** — https://refactoring.guru/design-patterns

6. **StackOverflow** — Specific questions about JavaFX module configuration in Eclipse.
7. **YouTube Videos for JDBC** — https://youtu.be/H9sN8KJYHkQ?si=Bu-44lu9Cn1vt5dL
8. **YouTube Videos for JavaFX** — https://youtu.be/VMOJ33mOOoc?si=3my7bTS-vnUv9L9h